BINF 4070 / COMS 4995 W008

The Future of Personal Health Assistant

Fall 2026

Searching Text: Keyword and Vector Search

Week 06

Dr. Xuhai “Orson” Xu | Dr. Eugene Wu

AI @ VP&S Initiative

Columbia | Vagelos College of Physicians and Surgeons

Collaboratory at Columbia University

DAP Lab

Today's Outline

1. **Searching Text**
2. **How Vector Search Works**
3. **Vector Search in DuckDB**
4. **Design Considerations**

Searching Text

Notes, and the questions people ask of them.

Notes as Data

note	patient	date	text
N1	P001	2026-03-15	Pt started on lisinopril 10 mg daily for HTN; A1c 8.2% on 3/15, up from 7.1%.
N3	P002	2026-02-10	Hives after amoxicillin in 2019. Penicillin allergy documented.
N4	P003	2026-04-01	No known drug allergies. Denies penicillin allergy.
N5	P004	2026-04-20	Mother has a penicillin allergy; patient tolerates amoxicillin.
N6	P002	2026-05-02	Rule out penicillin allergy; skin testing scheduled.
N8	P003	2026-06-11	Annual physical. Feels well. Labs pending.
N9	P006	2026-06-20	Allergic to PCN (rash as a child).

Which notes say the patient is allergic to penicillin?

Exact Match: LIKE

```
SELECT note_id FROM notes WHERE text ILIKE '%penicillin%';
```

Which notes does it return, and which should it?

note	patient	date	text	allergic?
N1	P001	2026-03-15	Pt started on lisinopril 10 mg daily for HTN; A1c 8.2% on 3/15, up from 7.1%.	
N3	P002	2026-02-10	Hives after amoxicillin in 2019. Penicillin allergy documented.	
N4	P003	2026-04-01	No known drug allergies. Denies penicillin allergy.	
N5	P004	2026-04-20	Mother has a penicillin allergy; patient tolerates amoxicillin.	
N6	P002	2026-05-02	Rule out penicillin allergy; skin testing scheduled.	
N8	P003	2026-06-11	Annual physical. Feels well. Labs pending.	
N9	P006	2026-06-20	Allergic to PCN (rash as a child).	

Exact Match: LIKE

```
SELECT note_id FROM notes WHERE text ILIKE '%penicillin%';
```

Highlighted: what `LIKE` matched. Which notes should it return?

note	patient	date	text	allergic?
N1	P001	2026-03-15	Pt started on lisinopril 10 mg daily for HTN; A1c 8.2% on 3/15, up from 7.1%.	no
N3	P002	2026-02-10	Hives after amoxicillin in 2019. Penicillin allergy documented.	yes
N4	P003	2026-04-01	No known drug allergies. Denies penicillin allergy.	**denied**
N5	P004	2026-04-20	Mother has a penicillin allergy; patient tolerates amoxicillin.	**mother**
N6	P002	2026-05-02	Rule out penicillin allergy; skin testing scheduled.	**unknown**
N8	P003	2026-06-11	Annual physical. Feels well. Labs pending.	no
N9	P006	2026-06-20	Allergic to PCN (rash as a child).	**yes**

`LIKE` returns N3–N6. A word can appear without the fact (N4, N5, N6), and the fact without the word (N9).

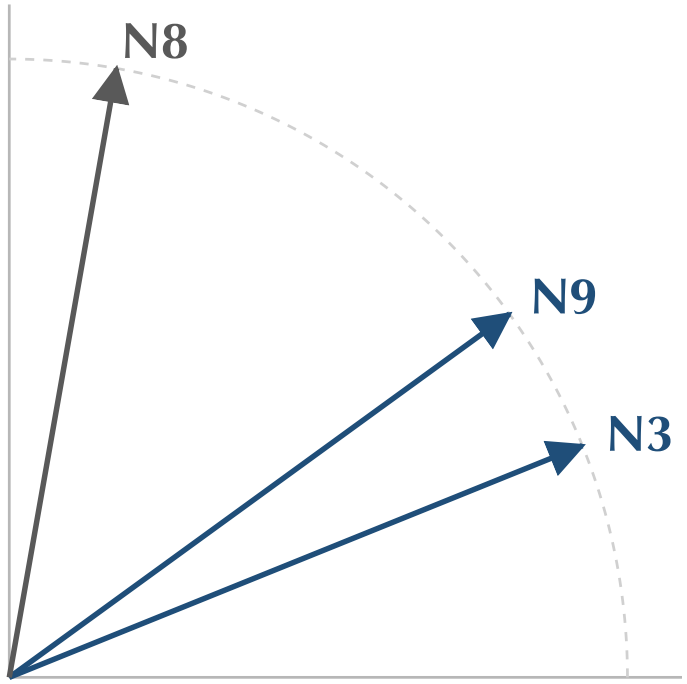
Three Ways to Search Text

	Keyword search	Semantic operator	Vector search
Match via	shared words	model over row + question	nearby vectors from a model
Model?s	never	every query and row	once per row on load, once per query
Cost/query	low	one model call per row	low, with an index
What it returns	rows ranked by word overlap	true or false per row	rows ranked by similarity

Vector search calls model when loading data. It's best effort similarity rather than reasoning.

Vector Search

Vector search represents each item and the query as vectors of numbers, then returns the **k** items whose vectors are nearest to the query's.



notes

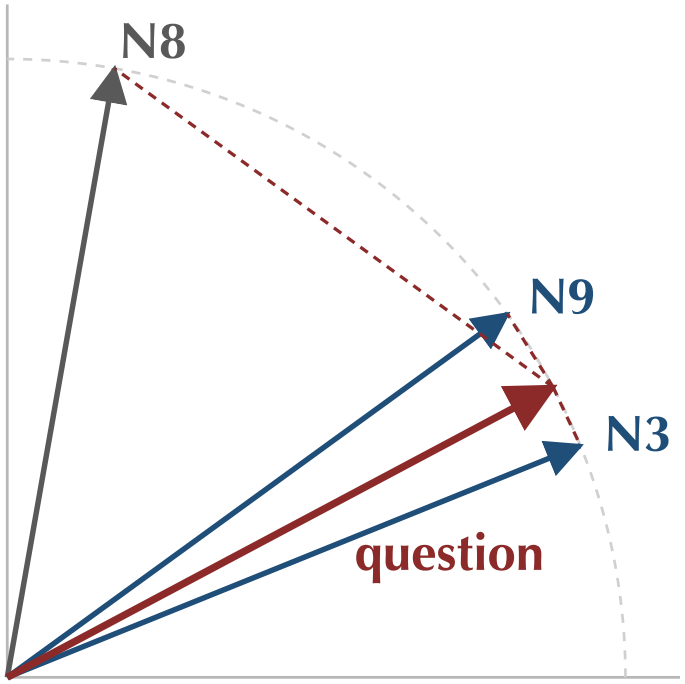
N3 penicillin allergy documented

N9 allergic to PCN

N8 annual physical

Vector Search

Vector search represents each item and the query as vectors of numbers, then returns the **k** items whose vectors are nearest to the query's.



notes

N3 penicillin allergy documented

N9 allergic to PCN

N8 annual physical

question "penicillin allergy"

note	cosine similarity	distance
N3	0.995	0.10
N9	0.990	0.14
N8	0.616	0.88

Distance Functions

For the query `q = (0.8, 0.6)`:

	vector	cosine similarity	Euclidean distance	inner product
A	(1, 0)	0.80	0.63	0.80
B	(0.6, 0.8)	**0.96**	**0.28**	0.96
C	(0, 1)	0.60	0.89	0.60

- Cosine for B: `(0.8 × 0.6 + 0.6 × 0.8) / (1 × 1) = 0.96`
- When every vector has length 1, the three give the same ranking; many embedding models, including OpenAI's, output length-1 vectors

Keyword Vectors and BM25

One dimension per word in the vocabulary; most entries are 0:

	penicillin	**allergy**	**allergic**	**pcn**	**amoxicillin**	**…**
query: “penicillin allergy”	1	1	0	0	0	
N3	1	1	0	0	1	
N9	0	0	1	1	0	

- **BM25** scores a note by the query words it contains, weighting rare words more and long notes less
- **Sparse:** one dimension per vocabulary word, so tens of thousands of dimensions, almost all 0
- **Exact words only:** N9 shares no word with the query and scores 0; systems usually expand the query with synonyms (PCN → penicillin) from a list or a terminology
- Exact, fast, and explainable: you can see which word matched

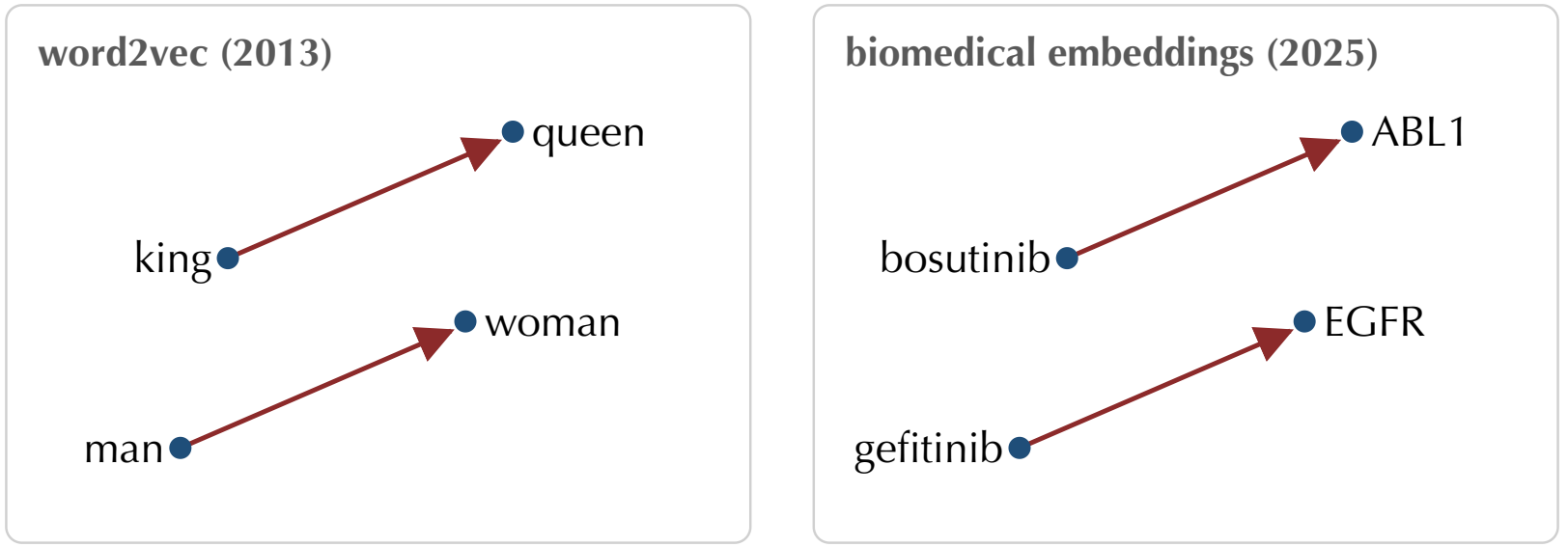
Learned Embeddings

An **embedding model** is a neural network trained so that texts with similar meaning get nearby vectors.

Model	Dimensions	Trained on
all-MiniLM-L6-v2	384	1 billion general sentence pairs
OpenAI small embedding	1,536	not disclosed
MedCPT	768	PubMed search logs

- **Dense:** a fixed number of dimensions set by the model, every entry nonzero: `"Allergic to PCN"` → `[0.021, −0.113, 0.087, …]`
- No single number means anything; only distances between vectors do
- Trained on related pairs, a model can place “PCN” near “penicillin” without a synonym list

Directions in Embedding Space



- **2013:** in word2vec, `king − man + woman` lands nearest `queen`
- **2024:** in LLaMA-2, concepts such as male→female and English→French are near-linear directions
- **2025:** in biomedical embeddings, drug + (average target − average drug) ranks the drug's true target first 30% of the time and in the top 10, 65%

Meaning has geometry, but only approximately.

Mikolov et al., NAACL (2013); Park et al., ICML (2024); Yamagiwa et al., Scientific Reports (2025)

What Embeddings Miss

Embeddings may miss **logical modifiers** that change a statement's meaning: negation, who it is about, certainty, time, and numbers.

Cosine similarity to "penicillin allergy":

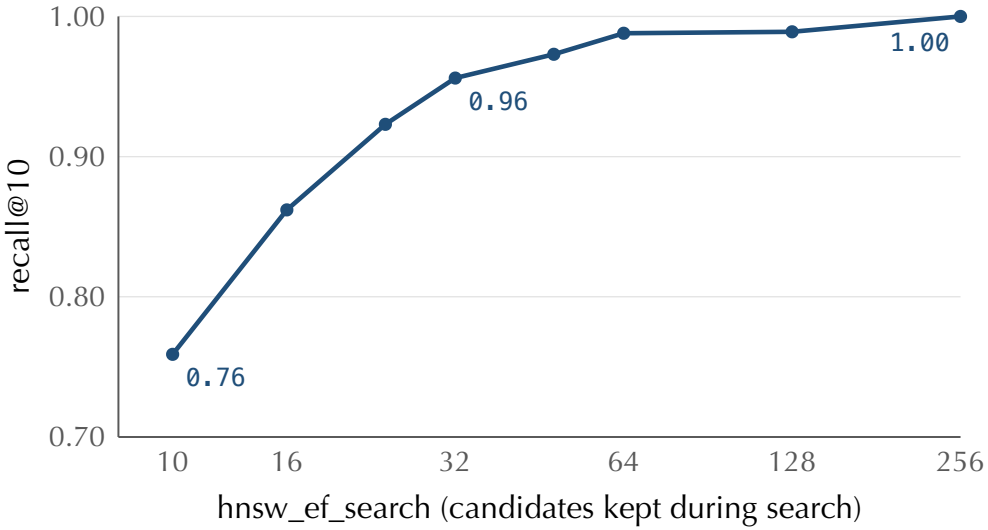
note_id	text	similarity	allergic?
N3	Hives after amoxicillin in 2019. Penicillin allergy documented.	0.992	yes
N9	Allergic to PCN (rash as a child).	0.990	yes
N4	No known drug allergies. Denies penicillin allergy.	0.938	no: denied
N5	Mother has a penicillin allergy; patient tolerates amoxicillin.	0.907	no: the mother

- On a negation benchmark, most retrieval models ranked no better than chance
- **Fix:** retrieve using vectors, then let a model decide

Exact and Approximate Search

- **Exact search** computes the distance to every row: 1,000,000 chunks × 384 numbers ≈ 384 million multiply-adds per query
- **Approximate nearest-neighbor (ANN)** search uses an index such as **HNSW**, a layered graph of near neighbors, and visits a small fraction of rows
- **Recall@10:** the share of the exact top 10 that the index returns; `SET hnsw_ef_search` trades speed for recall

DuckDB, 50,000 synthetic vectors.



Break

8 minutes

Next: vector search in DuckDB.

Vector Search in DuckDB

A vector column, a similarity query, an index, and keyword search.

Adding a Vector Column

notes.text → embedding model → notes.embedding

```
ALTER TABLE notes ADD COLUMN embedding FLOAT[1536];    -- fixed-length array type
ALTER TABLE notes ADD COLUMN embed_model TEXT;         -- which model made the vector
```

```
from openai import OpenAI
client = OpenAI()                                      # reads OPENAI_API_KEY
rows = con.sql("SELECT note_id, text FROM notes").fetchall()
resp = client.embeddings.create(model="text-embedding-3-small", input=[t for _, t in rows])
for (note_id, _), item in zip(rows, resp.data):        # one vector per input, already length 1
    con.execute("UPDATE notes SET embedding = ?, embed_model = ? WHERE note_id = ?",
                [item.embedding, "text-embedding-3-small", note_id])
```

Embedding Inside SQL

DuckDB has no built-in embedding function; extensions add one.

```
INSTALL flock FROM community; LOAD flock;
CREATE SECRET (TYPE OPENAI, API_KEY '...');           -- the key stays out of the query

SELECT note_id,
       llm_embedding({'model_name': 'text-embedding-3-small'},
                     {'context_columns': [{'data': text}]}) AS embedding
FROM notes;
```

Options in DuckDB

- flock extension (experimental)
- quackformers extension (experimental)
- MotherDuck (hosted DuckDB)

Flock, quackformers — DuckDB community extensions; MotherDuck embedding() documentation

Querying by Similarity

```
SELECT note_id, array_cosine_similarity(embedding, $q) AS sim   -- $q: the question's vector
FROM notes
WHERE note_date >= DATE '2026-04-01'                             -- ordinary SQL filters still apply
ORDER BY sim DESC
LIMIT 3;
```

With the toy vectors from the previous section:

note_id	note_date	sim
N9	2026-06-20	0.990
N6	2026-05-02	0.961
N4	2026-04-01	0.938

- `array_cosine_similarity`, `array_distance`, and `array_inner_product` are built in
- The query vector must come from the same model as the stored vectors

An HNSW Index in DuckDB

```
INSTALL vss; LOAD vss;
CREATE INDEX notes_hnsw ON notes USING HNSW (embedding) WITH (metric = 'cosine');
```

- Used for `ORDER BY` a distance function `LIMIT k`; the metric must match the function
- The index runs **first**: it returns the k nearest rows overall, then `WHERE` filters them

The previous slide's query, with the date filter:

without the index	with the index
N9, N6, N4	N9, N6

The index returned N3, N9, N6; the filter dropped N3 (February), leaving 2 rows instead of 3. Fixes: ask for more than k, filter first and search exactly, or keep one index per patient.

Keyword Search in DuckDB

```
INSTALL fts; LOAD fts;
PRAGMA create_fts_index('notes', 'note_id', 'text');          -- tokenize, stem, drop stop words

SELECT note_id, fts_main_notes.match_bm25(note_id, 'penicillin allergy') AS bm25
FROM notes WHERE bm25 IS NOT NULL ORDER BY bm25 DESC;
```

note_id	bm25	text
N4	0.60	No known drug allergies. Denies penicillin allergy.
N3	0.51	Hives after amoxicillin in 2019. Penicillin allergy documented.
N5	0.47	Mother has a penicillin allergy; patient tolerates amoxicillin.
N6	0.47	Rule out penicillin allergy; skin testing scheduled.

Hybrid Search: Reciprocal Rank Fusion

Combine the keyword and vector rankings: `score = 1 / (60 + keyword rank) + 1 / (60 + vector rank)`

```
WITH kw  AS (SELECT note_id, rank() OVER (ORDER BY bm25 DESC) AS r FROM keyword_hits),
     vec AS (SELECT note_id, rank() OVER (ORDER BY sim DESC)  AS r FROM vector_hits)
SELECT note_id, kw.r, vec.r, coalesce(1 / (60 + kw.r), 0) + 1 / (60 + vec.r) AS rrf
FROM vec LEFT JOIN kw USING (note_id) ORDER BY rrf DESC;
```

note_id	keyword rank	vector rank	rrf
N3	2	1	1/62 + 1/61 = 0.0325
N4	1	4	0.0320
N6	3	3	0.0317
N9	—	2	0.0161

Ranks are comparable across methods even though BM25 scores and cosines are not.

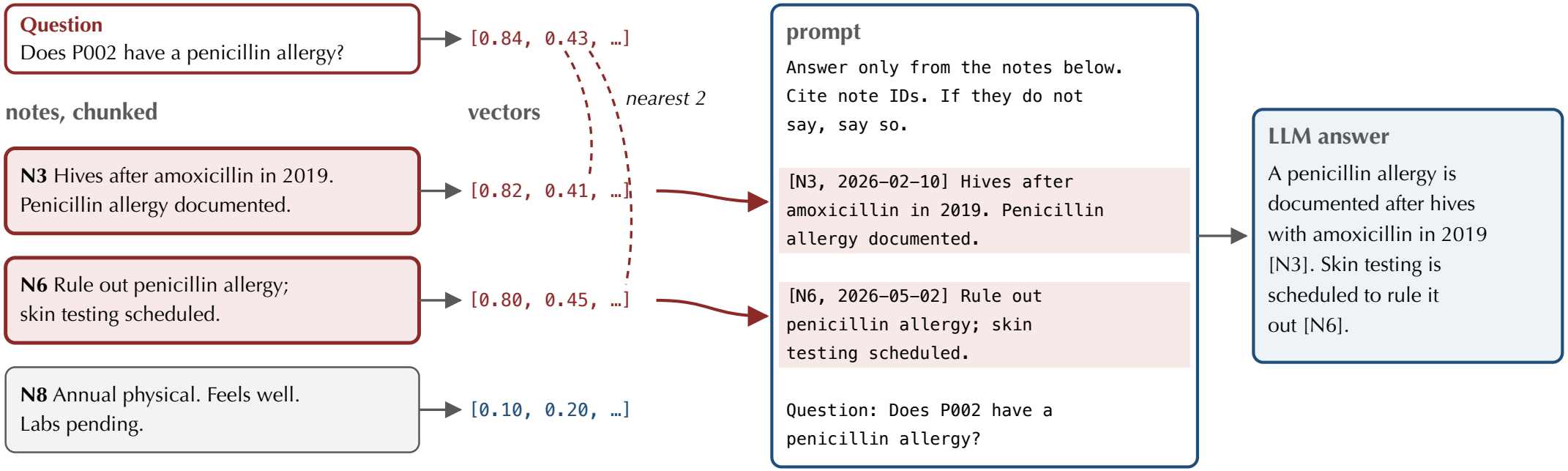
Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods — Cormack et al., SIGIR (2009)

The Same Search in Other Systems

System	Vector column	Nearest 3 to `$q`
DuckDB + `vss`	`FLOAT[384]`	`ORDER BY array_cosine_distance(embedding, $q) LIMIT 3`
PostgreSQL + pgvector	`vector(384)`	`ORDER BY embedding <=> $q LIMIT 3`
BigQuery	`ARRAY<FLOAT64>`	`VECTOR_SEARCH(TABLE notes, 'embedding', …, top_k => 3)`
turbopuffer	a `vector` field per document	`ns.query(rank_by=("vector", "ANN", q), limit=3)`

- pgvector: `<=>` is cosine distance, `<->` Euclidean, `<#>` negative inner product
- A dedicated vector database stores vectors and metadata; joins with the rest of your tables happen in your code

Retrieval-Augmented Generation



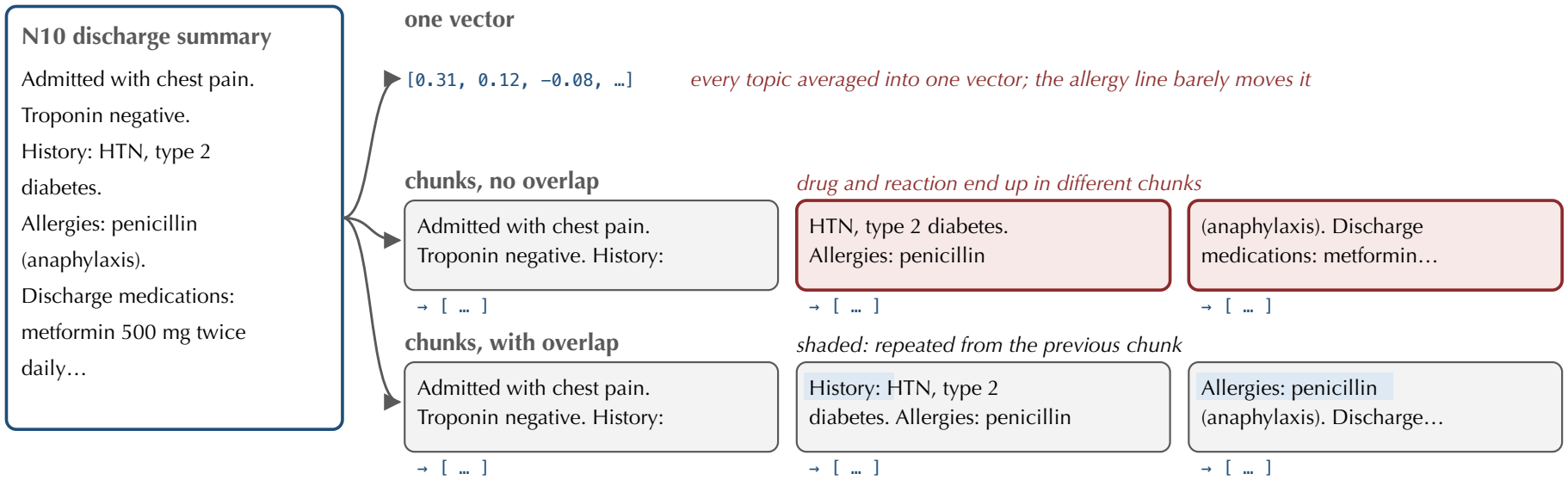
Retrieval-augmented generation (RAG) puts the retrieved passages into a language model's prompt and asks it to answer only from them.

- The model (hopefully) answers from retrieved data instead of its weights
- Relies on good retrieval!

Design Considerations

Chunking, models, filters, evaluation, and privacy.

Chunking



Choice	Common practice
Size	a few hundred tokens; smaller is more precise, larger keeps context
Overlap	10–20% of the chunk, so a fact cut at a boundary appears whole once
Boundaries	split at sentences, paragraphs, or section headings, not mid-word
Context	prepend note type, date, and section heading to each chunk

Evaluating Retrieval

A labeled set pairs each question with the notes that answer it:

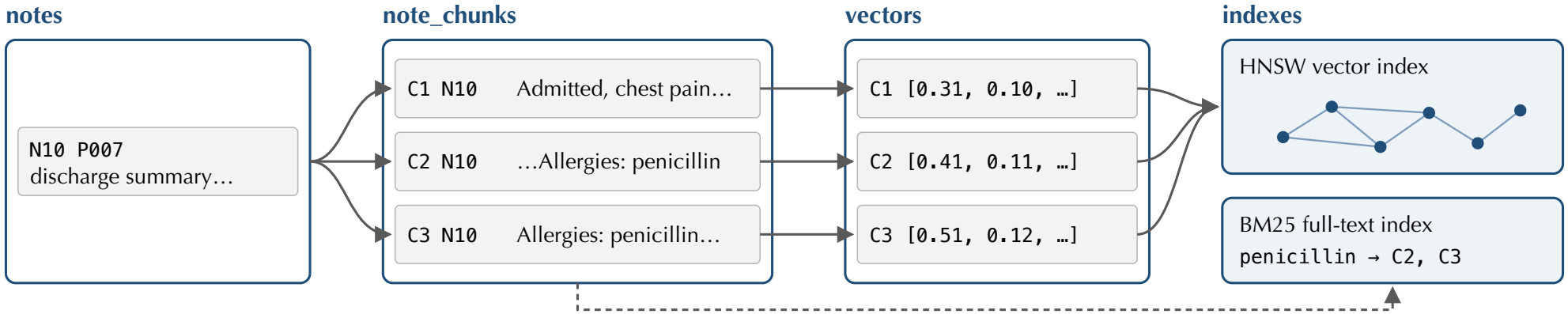
Question	Relevant notes
Is the patient allergic to penicillin?	N3, N9
What was the most recent HbA1c?	N1
Was a drug allergy denied or ruled out?	N4, N6

- **Precision@k:** % of top k that are relevant
- **Recall@k:** % of relevant notes in top k
- Average over all questions; build 20–50 real ones before choosing a model, chunk size, or k

Question 1, K=3:

Method	Top 3	Relevant in top 3	Precision@3	Recall@3
Keyword (BM25)	N4, N3, N5	N3	1/3 = 0.33	1/2 = 0.50
Vector (toy)	N3, N9, N6	N3, N9	2/3 = 0.67	2/2 = 1.00

Practical Matters



- **Vectors leak text:** an attack recovered 92% of 32-token inputs exactly from their embeddings, and 89% of full names in clinical notes. Protect vectors like the notes they came from
- **Deletion:** deleting a note must delete its chunks and vectors
- **Edits:** an edited note needs new vectors; an old vector still matches the old text
- **Indexes:** DuckDB's full-text index does not update when the table changes; rebuild it after loading new notes

Summary

Text needs its own search methods

Main Methods

- Keyword search: `ILIKE`, `LIKE` matches exactly.
- Vector search ranks by similarity
  - Vector indexes are used before `WHERE` clause filters, so may get fewer than expected result
- Query expansion often used to improve recall
- RAG retrieves chunks and passes to LLM for reasoning

Thanks for the Attention
Q&A